Práctica 04: Cortafuegos con iptables en un router Docker

Índice

1. Objetivo de la práctica **1**

2. Descripción del escenario **2**

3. Requisitos **3**

4. Pasos a seguir **3**
4.1. Completar el escenario . . . 3
4.2. Preparar el router . . . 4
4.3. Políticas por defecto y tráfico básico . . . 5
4.4. Publicar HTTP con DNAT . . . 5
4.5. Publicar MySQL con acceso restringido . . . 6
4.6. Guardar y automatizar las reglas . . . 7
4.7. Comprobaciones finales . . . 8

5. Diagnóstico de problemas **8**

6. Qué se valorará **9**

7. Instrucciones de entrega **9**

1. Objetivo de la práctica

Al finalizar la práctica, el alumno será capaz de:

- Desplegar con *Docker* una topología formada por una red externa, un router *Linux* y una red interna aislada.
- Comprobar que cada contenedor tiene su propia pila de red y sus propias tablas de `iptables`, independientes de las del host.
- Publicar los servicios de la red interna hacia la red externa mediante `DNAT`.
- Justificar y aplicar el `SNAT` necesario para que el tráfico publicado funcione en una red `internal` de *Docker*.
- Filtrar el tráfico con `iptables` usando políticas por defecto restrictivas y reglas de excepción.
- Restringir el acceso a un servicio (*MySQL*) a un único cliente autorizado.
- Registrar en el log del kernel los intentos de acceso bloqueados y revisarlos.
- Diagnosticar problemas de conectividad provocados por el cortafuegos.
- Documentar las reglas y automatizar su aplicación.

Está disponible la última versión de esta práctica en este enlace.

2. Descripción del escenario

Se parte del fichero docker-compose.yml incluido en esta carpeta. En él ya están definidos:

- La red `wan` (`172.30.0.0/24`), una red puente accesible desde el host, que representa la red real del host o la red externa del aula.
- La red `lan` (`192.168.50.0/24`), declarada como `internal: true`, de modo que *Docker* la aísla del exterior.
- El contenedor `router` (`ubuntu:24.04`), con dos interfaces:
  - `eth0` en `wan`, con IP `172.30.0.2`.
  - `eth1` en `lan`, con IP `192.168.50.2`.
- El reenvío IP activado en el router.

El host tiene la IP `172.30.0.1` en la red `wan` y actúa como cliente externo. Los servicios que hay que añadir en la red interna son:

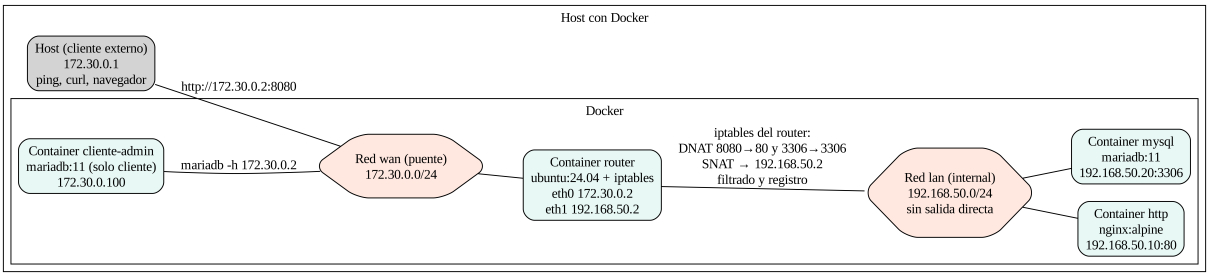
Servicio	Contenedor	IP	Puerto
HTTP	`http`	192.168.50.10	80/tcp
MySQL	`mysql`	192.168.50.20	3306/tcp

Además se añadirá un cliente externo de administración:

Cliente	Contenedor	IP
Administración	`cliente-admin`	172.30.0.100

El objetivo es que:

- Cualquier cliente de la red `wan` (por ejemplo, el propio host) pueda acceder al servidor HTTP mediante `http://172.30.0.2:8080`.
- Solo `cliente-admin` (`172.30.0.100`) pueda acceder al servidor MySQL en `172.30.0.2:3306`.
- Los intentos de acceso no autorizados a MySQL queden registrados en el log.
- Ningún otro tráfico atraviese el router hacia la red interna.



Nota: para que la práctica sea reproducible en cualquier equipo, la red `wan` se implementa como una red puente de *Docker* accesible desde el host. Si se dispone de una interfaz física y se quiere simular la red real, se puede sustituir por una red `macvlan`; las reglas de `iptables` del router son las mismas.

3. Requisitos

- *Docker* y *Docker Compose* instalados.
- Conexión a Internet para descargar las imágenes:
  - `ubuntu:24.04`
  - `nginx:alpine` (servidor HTTP)
  - `mariadb:11` (servidor *MySQL* compatible; la misma imagen incluye el cliente `mariadb`)
- Permisos para crear redes y contenedores. El router ya arranca con la capacidad `NET_ADMIN` y el reenvío IP activado.
- Un editor de texto y, en el host, un navegador o `curl`.

4. Pasos a seguir

4.1. Completar el escenario

Amplía el fichero `docker-compose.yml` para añadir los contenedores `http`, `mysql` y `cliente-admin`:

- `http`:
  - Imagen `nginx:alpine`.
  - IP fija `192.168.50.10` en la red `lan`.
  - Una página `index.html` personalizada, montada en `/usr/share/nginx/html/index.html`.
- `mysql`:
  - Imagen `mariadb:11`.
  - IP fija `192.168.50.20` en la red `lan`.
  - Variables `MARIADB_ROOT_PASSWORD`, `MARIADB_DATABASE`, `MARIADB_USER` y `MARIADB_PASSWORD`.
- `cliente-admin`:
  - Imagen `mariadb:11`, que ya incluye el cliente `mariadb`.
  - IP fija `172.30.0.100` en la red `wan`.
  - `entrypoint: ["sleep", "infinity"]`, para que no arranque ningún servidor y podamos usarlo como cliente.

Levanta el escenario y comprueba las redes y las IPs asignadas:

```
docker compose up -d
docker compose ps
docker network ls
docker network inspect 4_wan
docker network inspect 4_lan
```

El prefijo de las redes (`4`) depende del nombre del directorio del proyecto.

Comprueba el punto de partida antes de filtrar nada:

```
# El router es accesible desde el host
ping -c 2 172.30.0.2

# Todavía no hay nada publicado en el puerto 8080 del router
curl -m 3 http://172.30.0.2:8080/
```

La segunda orden falla con *connection refused*: el router no está escuchando en el 8080, pero todavía *contesta* con un rechazo. Más adelante, cuando el cortafuegos empiece a descartar paquetes, el fallo será por *timeout*: conviene notar la diferencia.

Comprueba también que la red `lan` está realmente aislada para los clientes externos: `cliente-admin` no puede llegar a los servidores aunque conozca sus IPs.

```
docker exec cliente-admin mariadb --connect-timeout=3 \
  -h 192.168.50.10 -P 3306 -u root -pnoexiste -e "SELECT 1;"
```

El cliente no consigue conectar: *Docker* descarta en el host el tráfico que, procedente de otras redes, se dirige hacia un puente `internal`. Solo queda el camino del router, que es lo que vamos a controlar.

4.2. Preparar el router

Entra en el router e instala las herramientas que necesitarás para el cortafuegos y para diagnosticar:

```
docker exec -it router bash
apt-get update
apt-get install -y iptables iproute2 iputils-ping curl tcpdump conntrack
```

Comprueba las interfaces, las rutas y el reenvío:

```
ip address
ip route
sysctl net.ipv4.ip_forward
```

Observa las tablas de `iptables` del router:

```
iptables -L -n -v
iptables -t nat -L -n -v
```

Están prácticamente vacías. Cada contenedor tiene su *propia* pila de red y sus propias tablas, independientes de las del host. Compáralas con las que *Docker* mantiene en el host (fuera del contenedor, con `sudo`):

```
sudo iptables -L -n -v
sudo iptables-save | grep -i docker
```

En el host verás cadenas como `DOCKER`, `DOCKER-FORWARD`, `DOCKER-INTERNAL` y `DOCKER-USER` (en versiones anteriores, `DOCKER-ISOLATION-STAGE-1` y `DOCKER-ISOLATION-STAGE-2`). Son las que *Docker* usa para gestionar sus puentes y aislar las redes internas, y conviene conocerlas porque explican parte del comportamiento del escenario.

4.3. Políticas por defecto y tráfico básico

Aplica políticas por defecto restrictivas y permite únicamente el tráfico imprescindible para el diagnóstico:

```
# Politicas por defecto
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT ACCEPT

# Loopback y conexiones ya establecidas
iptables -A INPUT -i lo -j ACCEPT
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

# Diagnostico: permitir ping al router desde la red externa
iptables -A INPUT -i eth0 -p icmp --icmp-type echo-request -j ACCEPT
```

Comprueba que el router sigue respondiendo a `ping` desde el host y que ahora el puerto 8080 ni siquiera rechaza la conexión:

```
ping -c 2 172.30.0.2
curl -m 3 http://172.30.0.2:8080/
```

Con `INPUT` y `FORWARD` en `DROP`, ningún tráfico atraviesa todavía el router hacia la red interna.

4.4. Publicar HTTP con DNAT

Publica el servidor HTTP de la red interna en el puerto `8080` del router. Una primera aproximación razonable sería hacer solo `DNAT` y permitir el reenvío:

```
# Redirigir el trafico que llega al router por el puerto 8080
iptables -t nat -A PREROUTING -i eth0 -d 172.30.0.2 \
  -p tcp --dport 8080 -j DNAT --to-destination 192.168.50.10:80

# Permitir el trafico reenviado hacia el servidor HTTP
iptables -A FORWARD -i eth0 -o eth1 -p tcp \
  -d 192.168.50.10 --dport 80 \
  -m conntrack --ctstate NEW,ESTABLISHED,RELATED -j ACCEPT
```

Comprueba desde el host:

```
curl -m 3 http://172.30.0.2:8080/
```

No funciona. Diagnostica el problema con `tcpdump` en el router mientras repites el `curl` desde el host:

```
tcpdump -ni eth1 tcp port 80
```

Ves los `SYN` saliendo por `eth1` hacia `192.168.50.10`, pero nunca vuelve ninguna respuesta: el paquete sale del router y no llega al servidor. La explicación está en el host: mira las reglas que *Docker* instala para la red interna.

```
sudo iptables-save | grep -A5 DOCKER-INTERNAL
# o, en versiones anteriores de Docker:
sudo iptables-save | grep -A5 DOCKER-ISOLATION-STAGE-1
```

Entre las reglas aparecen dos como estas (donde `br-XXXX` es el puente de la red `lan`):

```
-A DOCKER-INTERNAL ! -s 192.168.50.0/24 -o br-XXXX -j DROP
-A DOCKER-INTERNAL ! -d 192.168.50.0/24 -i br-XXXX -j DROP
```

Docker descarta en el host cualquier paquete que circule por el puente de una red `internal` con una IP de origen o de destino ajena a la subred. El paquete que hemos reenviado llega al puente con su origen original (`172.30.0.1`), así que el host lo descarta antes de que alcance al contenedor `http`.

La solución es que el router *enmascare* ese tráfico con su propia IP de la red interna:

```
# SNAT del trafico que el router reenvia hacia la lan
iptables -t nat -A POSTROUTING -o eth1 -d 192.168.50.0/24 \
  -j SNAT --to-source 192.168.50.2
```

Ahora el paquete entra en el puente con origen `192.168.50.2` (una IP de la subred), y la respuesta del servidor vuelve directamente al router, que restaura las IPs originales gracias a `conntrack`. Compruébalo:

```
curl -v http://172.30.0.2:8080/
```

Debe aparecer la página personalizada. Comprueba también que el puerto 80 del router *no* está publicado:

```
curl -m 3 http://172.30.0.2:80/
```

Como efecto secundario del `SNAT`, el servidor HTTP ve las conexiones como si vinieran del router. Observa el log de accesos:

```
docker logs http
```

Y revisa en el router las entradas de `conntrack` de la conexión que acabas de hacer:

```
conntrack -L | grep 192.168.50
```

Nota: en un router físico, con los servidores usando al router como puerta de enlace, este `SNAT` no sería necesario. En este escenario hace falta precisamente por el aislamiento que *Docker* aplica a las redes `internal`. Es el precio de que el host controle el acceso a la red interna y de que los servidores no necesiten ruta por defecto hacia el router: las respuestas vuelven a él por estar en la misma subred.

4.5. Publicar MySQL con acceso restringido

Publica MySQL en el puerto `3306` del router, pero permite el acceso únicamente al cliente de administración:

```
# Redirigir el trafico del puerto 3306 al servidor MySQL
iptables -t nat -A PREROUTING -i eth0 -d 172.30.0.2 \
  -p tcp --dport 3306 -j DNAT --to-destination 192.168.50.20:3306

# Permitir solo al cliente 172.30.0.100
iptables -A FORWARD -i eth0 -o eth1 -s 172.30.0.100 \
  -p tcp -d 192.168.50.20 --dport 3306 \
  -m conntrack --ctstate NEW,ESTABLISHED,RELATED -j ACCEPT

# Registrar y bloquear el resto de intentos
iptables -A FORWARD -i eth0 -o eth1 -p tcp \
  -d 192.168.50.20 --dport 3306 \
  -j LOG --log-prefix "MYSQL-DENEGADO: " --log-level 4
iptables -A FORWARD -i eth0 -o eth1 -p tcp \
  -d 192.168.50.20 --dport 3306 -j DROP
```

Fíjate en el orden: la regla que acepta a `cliente-admin` debe estar *antes* que las de `LOG` y `DROP`. El destino `LOG` no bloquea nada por sí mismo: solo registra el paquete y deja que siga evaluando reglas.

La regla de `SNAT` del paso anterior también se aplica a estas conexiones, así que el procedimiento es el mismo que con HTTP.

Comprueba el acceso desde `cliente-admin`:

```
docker exec cliente-admin mariadb --connect-timeout=5 \
  -h 172.30.0.2 -P 3306 -u alumno -palumno1234 \
  -e "SELECT VERSION();"
```

Debe responder con la versión de *MariaDB*. Comprueba desde el host que el acceso está bloqueado:

```
timeout 5 bash -c 'cat < /dev/null > /dev/tcp/172.30.0.2/3306'
echo $?
```

El comando debe terminar con un código distinto de `0` y agotar el *timeout* (un `DROP` no rechaza la conexión: la silencia). Revisa los intentos bloqueados en el log del kernel del host (el kernel es compartido entre el host y los contenedores, así que los mensajes de `iptables -j LOG` del router aparecen ahí):

```
sudo dmesg | grep MYSQL-DENEGADO
# o, si tu sistema usa journald:
sudo journalctl -k | grep MYSQL-DENEGADO
```

4.6. Guardar y automatizar las reglas

Las reglas de `iptables` se pierden si el contenedor se reinicia. Reúnelas todas en un script `router/firewall.sh`, móntalo en el router con un volumen y ejecútalo tras levantar el escenario:

```
router:
  volumes:
    - ./router/firewall.sh:/firewall.sh:ro
```

```
docker exec router bash /firewall.sh
```

Como alternativa puedes volcar y restaurar las reglas con `iptables-save` e `iptables-restore` (fíjate en la opción `-i` para pasar el fichero por la entrada estándar):

```
docker exec router iptables-save > router/rules.v4
docker exec -i router iptables-restore < router/rules.v4
```

Incluye el script en la entrega y explica qué harías para que las reglas se aplicaran automáticamente al recrear el escenario.

4.7. Comprobaciones finales

Rellena una tabla como esta con el resultado real de cada prueba:

Origen	Destino	Puerto	Resultado esperado
host 172.30.0.1	172.30.0.2	icmp	Permitido (ping)
host 172.30.0.1	172.30.0.2	8080/tcp	Permitido (HTTP)
host 172.30.0.1	172.30.0.2	80/tcp	Bloqueado
host 172.30.0.1	172.30.0.2	3306/tcp	Bloqueado (y en el log)
cliente-admin 172.30.0.100	172.30.0.2	3306/tcp	Permitido (MySQL)
cliente-admin 172.30.0.100	192.168.50.10	cualquiera	Bloqueado (red aislada)

Incluye la salida de:

```
docker exec router iptables -L -n -v
docker exec router iptables -t nat -L -n -v
docker exec router conntrack -L
```

y capturas de las pruebas permitidas y bloqueadas.

5. Diagnóstico de problemas

Responde brevemente a estas cuestiones:

1. Desde el host no se accede a `http://172.30.0.2:8080`. Describe cómo lo diagnosticarías con `tcpdump` en el router y qué revisarías en las tablas `nat` y `filter`.
2. `cliente-admin` no puede conectar con MySQL, pero el `ping` del host al router sí funciona. ¿Qué regla falta o está mal colocada?
3. Las conexiones al servidor HTTP llegan a `http` con dirección de origen `192.168.50.2`. ¿Por qué? ¿Qué implicación tiene esto para el servidor?
4. Aunque añadas una regla de `MASQUERADE` en el router, los contenedores de la `lan` no consiguen salir a Internet. Investiga con `sudo iptables-save` qué reglas instala *Docker* en el host para las redes internas y explica el motivo.
5. Al reiniciar el contenedor `router` se pierden las reglas. ¿Cómo lo solucionas para que el escenario quede protegido nada más arrancar?
6. Comprueba si desde el host puedes acceder directamente a `http://192.168.50.10` sin pasar por el router, y compara con lo que le ocurre a `cliente-admin` al intentarlo. ¿Qué indica esto sobre dónde aplican las reglas de aislamiento de *Docker* y por qué el host es un caso especial?

6. Qué se valorará

Se entregará un documento (`PDF`, `ODT` o `DOCX`) con:

- **Topología:** diagrama con las redes, contenedores, IPs y flujo del tráfico.
- **Despliegue:** `docker-compose.yml` completo y comandos utilizados.
- **Cortafuegos:** script `firewall.sh` con las reglas comentadas y explicación de cada una, incluida la justificación del `SNAT`.
- **Comprobaciones:** tabla de pruebas, salidas de `iptables` y `conntrack`, y capturas de los accesos permitidos y bloqueados.
- **Logs:** captura de los intentos de acceso a MySQL bloqueados.
- **Diagnóstico:** respuestas razonadas a los problemas planteados.
- **Claridad y presentación:** pasos ordenados, explicaciones propias y capturas legibles.

7. Instrucciones de entrega

- **Modalidad:** individual. Solo se admitirá en pareja si así se indica expresamente en clase.
- **Lugar de entrega:** tarea correspondiente a esta práctica en el aula virtual del módulo.
- **Contenido:** documento de la práctica, `docker-compose.yml`, `firewall.sh` e `index.html`.
- **Fecha:** la indicada en el aula virtual.